Interim Report

Amna Malik

Caden Craddock

Matthew Reyna

Texas Tech University

CS2365: Object-Oriented Programming

4/22/2026

Table of Contents

Introduction 3

UML / Class Diagram 4

OOP Design Plan 5

- Abstraction / Encapsulation 5
- Interface 5
- Abstract Class 5
- Inheritance 5
- Exception Handling 6

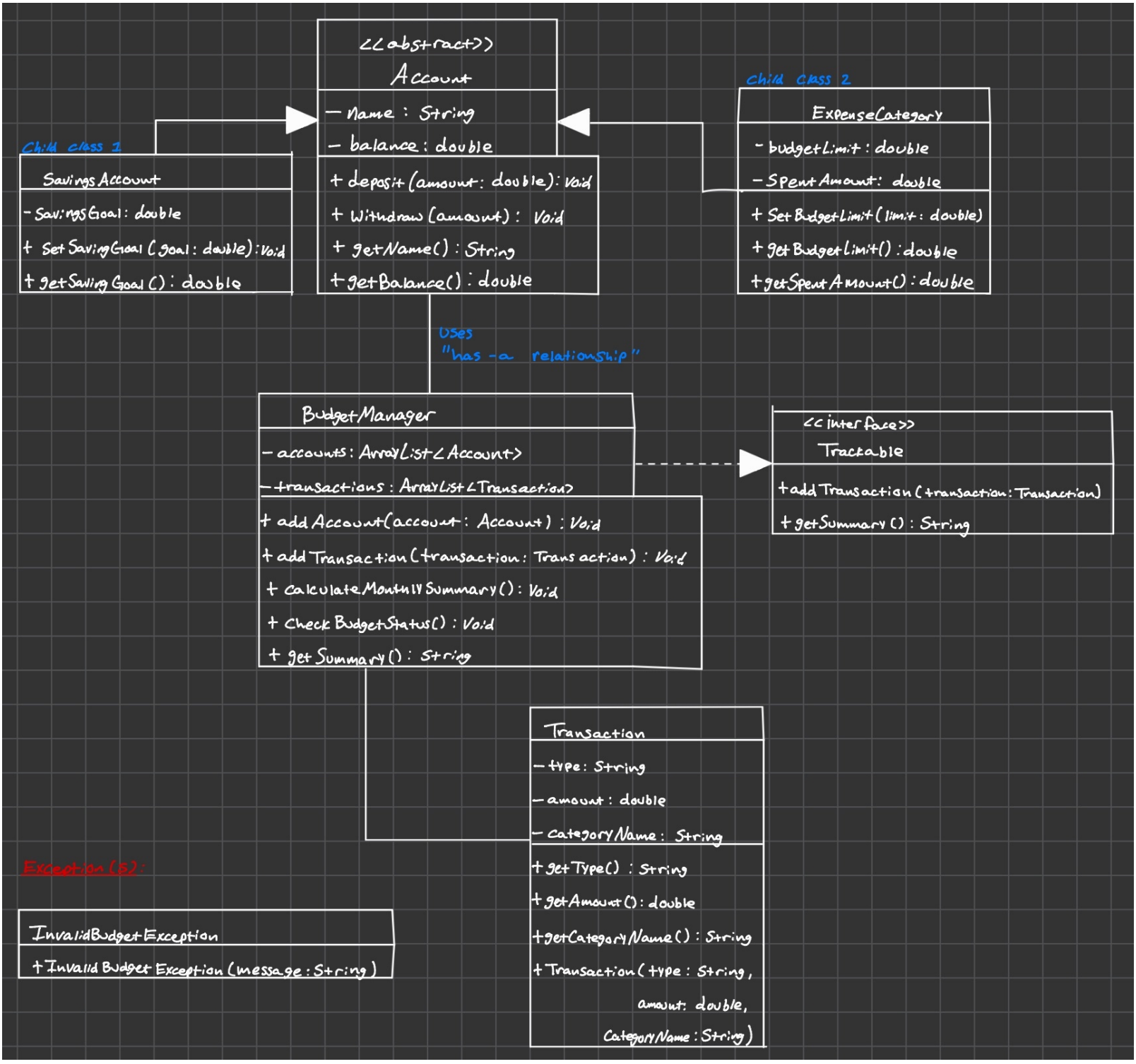
Group Member Roles 7

- Caden Craddock – Member 1: Core Class Design and OOP Structure 7
- Matthew Reyna – Member 2: Transaction Logic and Exception Handling 7
- Amna Malik – Member 3: Budget Limits, Monthly Summary, and Output 7

Introduction

The purpose of this project is to develop a Personal Finance / Budget Tracker using object-oriented programming concepts in Java. The program will allow users to track income and expenses, organize financial data into categories, and generate monthly summaries. This project focuses on applying key OOP concepts such as abstraction, encapsulation, interfaces, inheritance, and exception handling in a structured and practical way.

UML / Class Diagram



OOP Design Plan

Abstraction / Encapsulation

Our project uses abstraction by separating the finance tracker into different classes, each with its own responsibility. For example, **Account** stores general account information, **Transaction** stores transaction details, and **BudgetManager** manages the overall budget system. Encapsulation is used by keeping important data, such as balances, budget limits, and transaction information, inside private class variables and allowing access through public methods. This helps protect the data and keeps the program organized.

Interface

Our project uses an interface called **Trackable**. This interface defines common behaviors related to financial tracking, such as adding a transaction and generating a summary. By using an interface, we make sure that any class responsible for tracking financial information follows the same structure. This improves consistency and demonstrates the use of interfaces in object-oriented programming.

Abstract Class

Our project uses an abstract class called **Account**. This class contains the common fields and methods shared by account-related classes, such as the account name, balance, deposit, and withdraw methods. Since the program does not need to create a generic account directly, the class is abstract. This allows child classes to inherit shared behavior while still having their own specialized features.

Inheritance

Inheritance is used by creating child classes such as **SavingsAccount** and **ExpenseCategory** from the parent class **Account**. These child classes inherit common fields and methods from **Account**, which

reduces repeated code. Each child class also includes its own unique features, such as a savings goal or a budget limit. This creates a clear class hierarchy in the project.

Exception Handling

Exception handling will be used to make the program more reliable when the user enters invalid data. For example, the project includes a custom exception called **InvalidBudgetException**, which can be used when a user enters an invalid budget amount or another incorrect financial value. This helps prevent the program from crashing and allows errors to be handled in a clear way.

Group Member Roles

Our group divided the Personal Finance / Budget Tracker project into three main parts so that each member could contribute to both the functionality of the program and the required object-oriented programming concepts.

Caden Craddock – Member 1: Core Class Design and OOP Structure

Responsible for designing the main classes used in the program, including **Account**, **SavingsAccount**, and **ExpenseCategory**. This member implements encapsulation through private variables and public methods, creates the **Trackable** interface, and establishes inheritance relationships between parent and child classes. This role ensures that abstraction, encapsulation, and inheritance are properly demonstrated in the project.

Matthew Reyna – Member 2: Transaction Logic and Exception Handling

Responsible for implementing the logic for tracking income and expenses, updating balances, and managing transactions. This member connects transactions to accounts and categories and implements exception handling, including the use of the **InvalidBudgetException** class to handle invalid input or incorrect financial data.

Amna Malik – Member 3: Budget Limits, Monthly Summary, and Output

Responsible for implementing budget limit features, checking whether spending is over or under budget, and generating the monthly summary of income, expenses, and remaining balance. This member also prepares program outputs, test runs, and presentation-ready demonstrations of the project, focusing on the user-facing side of the application.